



CICLO FORMATIVO DE GRADO SUPERIOR EN Desarrollo de Aplicaciones Multiplataforma

PROYECTO FIN DE CICLO

Título: Hanechi NoteBook

Alumno: Alejandro Dastard Dore

Nº Alumno: 356026

DECLARACIÓN JURADA DE AUTORÍA

D. Alejandro Dastard Dore, con N.I.F. 54368041C, como autor de este documento académico, titulado: Documentación PFC Hanechi y presentado como Proyecto Fin de Ciclo para la obtención del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma,

DECLARO QUE

Soy el único autor del trabajo, con la excepción de referencias a contenidos o ideas de otros autores, en cuyo caso han sido explícitamente citados.

El trabajo remitido es un documento original, que no ha sido publicado, ya sea total o parcialmente, ni presentado para obtención de un título académico en ninguna institución académica u organización.

No he transgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual.

Soy consciente de que el hecho de no respetar estos extremos es una falta grave de integridad académica y podrá ser objeto de sanciones.

En Pozuelo de Alarcón (Comunidad de Madrid), a 27 de Abril de 2026

A handwritten signature in black ink, consisting of a large, stylized 'A' followed by a horizontal line and a small flourish.

(firma)

Fdo.: Alejandro Dastard Dore

RESUMEN

Hanechi NoteBook, es una aplicación móvil desarrollada para llenar el hueco entre notas simples y complejidad de diseño. Su objetivo principal es que cualquier usuario pueda crear cuadernos visualmente atractivos de forma intuitiva, sin necesidad de conocimientos previos. Para ello, ofrece un sistema de edición basado en un lienzo dinámico con bloques móviles que permite organizar cada página.

Además, permite la fácil difusión de contenido mediante su entorno social. Inspirado en Pinterest, cuenta con un feed de exploración donde los usuarios pueden compartir sus obras, inspirarse en el trabajo de otros e interactuar mediante un sistema de "likes", guardados y seguimiento de creadores.

Para este proyecto, he realizado una arquitectura con React Native para la interfaz móvil y Java con Spring Boot para el servidor, gestionando la persistencia de los datos con PostgreSQL. El sistema garantiza la integridad y sincronización de la información y cuenta con un flujo de seguridad que incluye autenticación por tokens (JWT) y verificación de dos factores (2FA) por correo electrónico, asegurando que el contenido de cada usuario esté siempre protegido.

ABSTRACT

Hanechi NoteBook is a mobile application developed to bridge the gap between simple notes and design complexity. Its primary goal is to allow any user to intuitively create visually appealing notebooks without requiring prior layout skills. To achieve this, it offers an editing system based on a dynamic canvas with movable blocks, allowing users to organize each page.

Additionally, it facilitates easy content sharing through its integrated social environment. Inspired by Pinterest, it features an exploration feed where users can share their work, find inspiration from others, and interact using a system of likes, saves, and following creators.

For this project, I have developed an architecture using React Native for the mobile interface and Java with Spring Boot for the server, managing data persistence with PostgreSQL. The system guarantees information integrity and synchronization, and includes a security workflow with token-based authentication (JWT) and two-factor verification (2FA) via email, ensuring each user's content remains fully protected at all times.

Contenido

1. INTRODUCCIÓN	5
1.1. Motivación del Proyecto	5
1.2. Problema Actual	5
1.3. Propuesta de Solución	5
2. REQUISITOS FUNCIONALES	6
2.1 Edición Creativa y Personalización	6
2.2 Apartado Social y Descubrimiento	6
2.3 Gestión de Comunidad y Persistencia	7
3. DISEÑO DE LA SOLUCIÓN	8
3.1. Mapa de navegación	8
3.1.1. Acceso y Seguridad	9
3.1.2. Inicio	9
3.1.3. Cuadernos	10
3.1.4. Menú Lateral	11
3.2. Modelo de base de datos	13
Módulo de Usuario	13
Módulo de Contenido	14
Módulo Social	14
Módulo de Estadísticas	14
Tabla de Relaciones	15
3.3. Diseño de la arquitectura	16
3.4. Lenguajes utilizados	17
4. DISEÑO TÉCNICO	17
4.1. Libro de estilo	17
4.1.1. Rejilla utilizada	17
4.1.2. Logotipo	18
4.1.3. Tipografía	19
4.1.4. Paleta de colores	19
4.1.5. Imágenes y gráficos	20
4.1.6. Estilo de componentes	20
4.2. Definición de componentes	21
4.2.1. Gestión de Acceso y Seguridad	21
4.2.2. Gestión de Contenido y Edición	22
4.2.3. Gestión de Perfiles	22
4.2.4. Gestión Social y de Estadísticas	23
4.2.5. Estado Global (Zustand)	23
4.2.6. Servicio de Notificación MFA vía EmailService	24
4.3. Diseño de tablas de la base de datos	25
5. DEMOSTRACIÓN	27
5.1. Estructura del Proyecto	27
5.2. Requisitos del Entorno	27
5.3. Configuración y Puesta en Marcha	27

Paso 1: Clonar el repositorio	27
Paso 2: Base de Datos y Backend	27
Paso 3: Inserción de Datos Iniciales	28
Paso 4: Configuración del Frontend	28
6. BIBLIOGRAFÍA	28
7. ANEXOS	29
7.1. Código fuente	29
7.2.1. Backend (/backend)	29
7.2.2. Base de Datos (/database)	37
7.2.3. Frontend (/frontend)	37

ÍNDICE DE ILUSTRACIONES

Ilustración 1 : Mapa de Navegación.....	8
Ilustraciones 2-4 : Módulo de Cuaderno.....	9
Ilustraciones 5-7 : Módulo de Inicio.....	10
Ilustraciones 8-10 : Módulo de Cuaderno.....	11
Ilustraciones 11-13 : Menú Lateral.....	12
Ilustración 14 : Mapa de Base de Datos. Fuente: dbdiagram.io.....	13
Ilustraciones 15-18 : Estructura modular de la API.....	17
Ilustración 19 : Rejilla y Métricas de AppTheme.js.....	18
Ilustración 20 : Logotipo.....	18
Ilustración 21 : Paleta de colores de AppTheme.js.....	21
Ilustración 22 : Correo Verificación.....	25

ÍNDICE DE TABLAS

Tabla 1 : Relaciones.....	16
Tabla 2: Tipografía de AppTheme.js.....	19
Tabla 3: Módulo de Estadísticas. Fuente: Base de Datos (Usuario).....	25
Tabla 4: Módulo de Estadísticas. Fuente: Base de Datos (Contenido).....	26
Tabla 5: Módulo de Estadísticas. Fuente: Base de Datos (Social).....	26
Tabla 6: Módulo de Estadísticas. Fuente: Base de Datos (Estadísticas).....	26

1. INTRODUCCIÓN

En la actualidad, el uso del móvil para tomar apuntes ha dejado casi obsoleto al papel. Sin embargo, con las herramientas disponibles existe un vacío entre las aplicaciones de notas simples y las plataformas de diseño complejas. **Hanechi NoteBook** nace para llenar ese hueco, ofreciendo un espacio donde la creatividad y la comunidad van de la mano.

El nombre combina dos conceptos del japonés: Hane (羽 - Pluma) y Chi (地 - Tierra). Representando de esta forma la unión entre la herramienta de escritura y el sistema social.

1.1. Motivación del Proyecto

Muchas personas quieren que sus notas se vean bien, pero no tienen el tiempo o los conocimientos para usar herramientas de maquetación profesional. Este proyecto busca que cualquier usuario pueda crear cuadernos visualmente atractivos de forma intuitiva.

1.2. Problema Actual

Tras investigar soluciones como Google Drive, Keep o Notion, se han identificado tres problemas clave que afectan al usuario medio:

- **Rigidez en la edición:** Las apps actuales suelen ser listas de texto plano o bloques fijos que no permiten "jugar" con la disposición de los elementos en pantalla.
- **Aislamiento:** El contenido suele morir en el dispositivo del autor. No hay una forma sencilla de inspirarse en el trabajo de otros usuarios dentro de la misma herramienta.
- **Apps complejas:** Las herramientas con más funciones tienen curvas de aprendizaje muy altas, lo que desmotiva al usuario que solo quiere algo rápido y bonito.

1.3. Propuesta de Solución

Hanechi NoteBook propone un modelo equilibrado que se apoya en tres pilares:

1. **Edición Creativa:** Un sistema basado en bloques móviles (títulos, imágenes, listas) que permite organizar cada página como si fuera un lienzo.

2. **Ecosistema Social:** Un muro visual (estilo Pinterest) donde compartir y descubrir cuadernos públicos para fomentar la inspiración colectiva.
3. **Tecnología Robusta:** Una arquitectura moderna usando React Native y Spring Boot que garantiza que los datos estén siempre seguros y sincronizados.

2. REQUISITOS FUNCIONALES

2.1 Edición Creativa y Personalización

Este módulo constituye el núcleo de la herramienta, proporcionando un entorno flexible para la expresión visual.

- **Creación y Gestión de Cuadernos:** El usuario podrá iniciar nuevos proyectos desde cero, asignarles un título, una categoría y personalizar la portada.
- **Lienzo Dinámico:** Interfaz de edición que permite la inserción de diferentes elementos sobre las páginas del cuaderno.
- **Interacción Drag and Drop:** Sistema detecta y reorganiza los elementos que pueden ser arrastrados fácilmente gracias a la librería de draggable flatlist.
- **Organización de Páginas:** Herramientas para añadir nuevas hojas, eliminar páginas existentes o reordenar la secuencia del cuaderno de forma visual.
- **Guardado en Tiempo Real:** El sistema asegura que cada cambio realizado en la edición se sincronice automáticamente para evitar la pérdida de progreso.

2.2 Apartado Social y Descubrimiento

Inspirado en la navegación visual, este apartado fomenta el intercambio de ideas entre los miembros de la plataforma.

- **Exploración Estilo Pinterest:** Feed principal donde se muestran miniaturas de los cuadernos con su nombre y código único, facilitando la búsqueda de contenido deseado mediante filtros.

- **Interacción con Contenido:** Los usuarios podrán marcar cuadernos como favoritos y guardarlos en tableros personales o colecciones de "Inspiración" para su posterior consulta.
- **Visualización de Proyectos:** Modo de lectura a pantalla completa que permite hojear los cuadernos de otros usuarios sin interferir en su diseño original.

2.3 Gestión de Comunidad y Persistencia

Funciones destinadas a la administración de usuarios y la estabilidad de la información en el ecosistema.

- **Sistema de Conexiones:** Capacidad para buscar y agregar amigos, así como seguir a creadores específicos.
- **Perfiles de Usuario:** Espacio personal donde se gestionan los cuadernos propios (públicos y privados), la lista de amigos y las estadísticas básicas de interacción.
- **Notificaciones de Actividad:** Avisos dentro de la aplicación sobre nuevas interacciones sociales, como nuevos seguidores o información sobre cuadernos.
- **Gestión de Base de Datos:** Integración total con el backend para garantizar que la estructura de los cuadernos, los elementos decorativos y las relaciones sociales se mantengan persistentes y seguras en cada sesión.

3. DISEÑO DE LA SOLUCIÓN

Basado en los requisitos funcionales, en esta sección se especifica el diseño que se plantea del proyecto para asegurar que dará respuesta a las funcionalidades planteadas.

3.1. Mapa de navegación

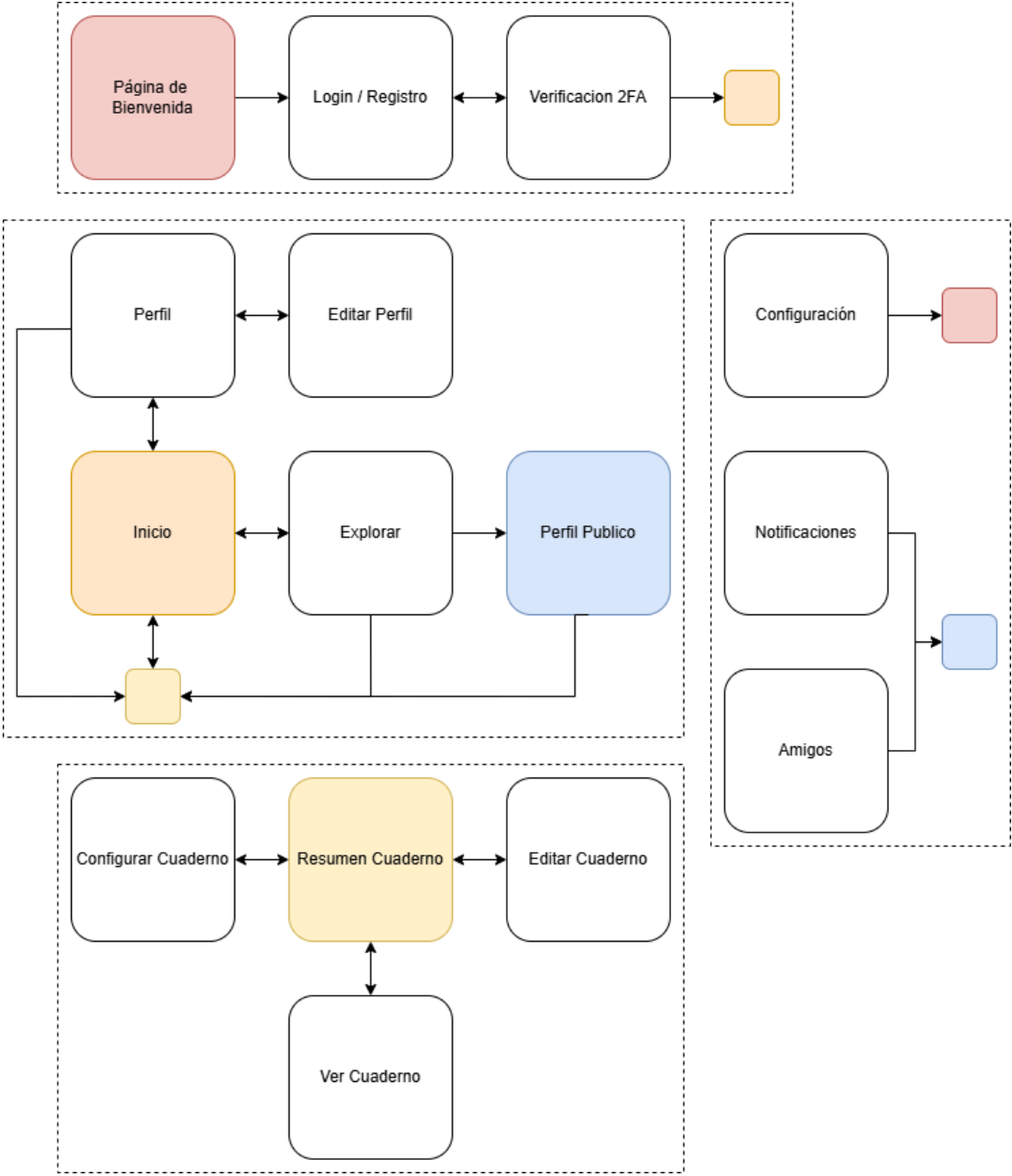
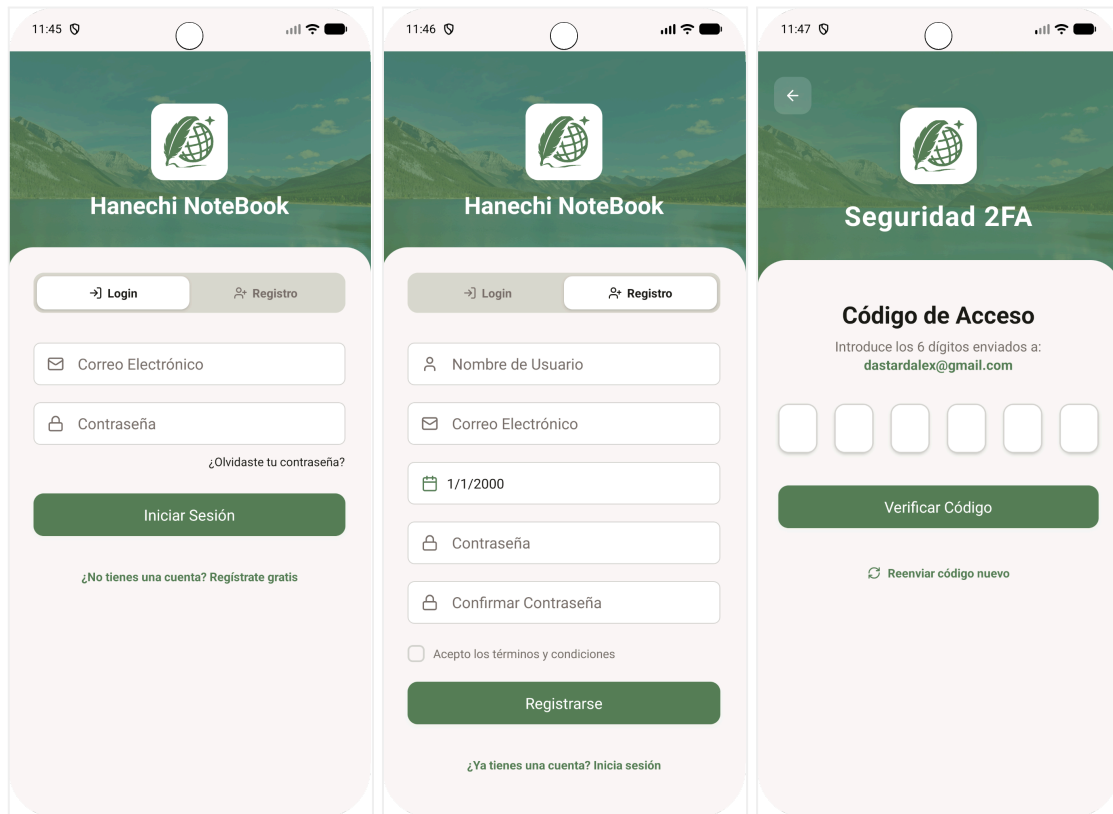


Ilustración 1 : Mapa de Navegación

3.1.1. Acceso y Seguridad

El usuario inicia su sesión mediante Registrarse o Iniciar Sesión. Cuenta como métodos de seguridad el uso de JWT y un sistema de autenticación de dos factores enviado al correo electrónico del usuario.



Ilustraciones 2-4 : Módulo de Cuaderno

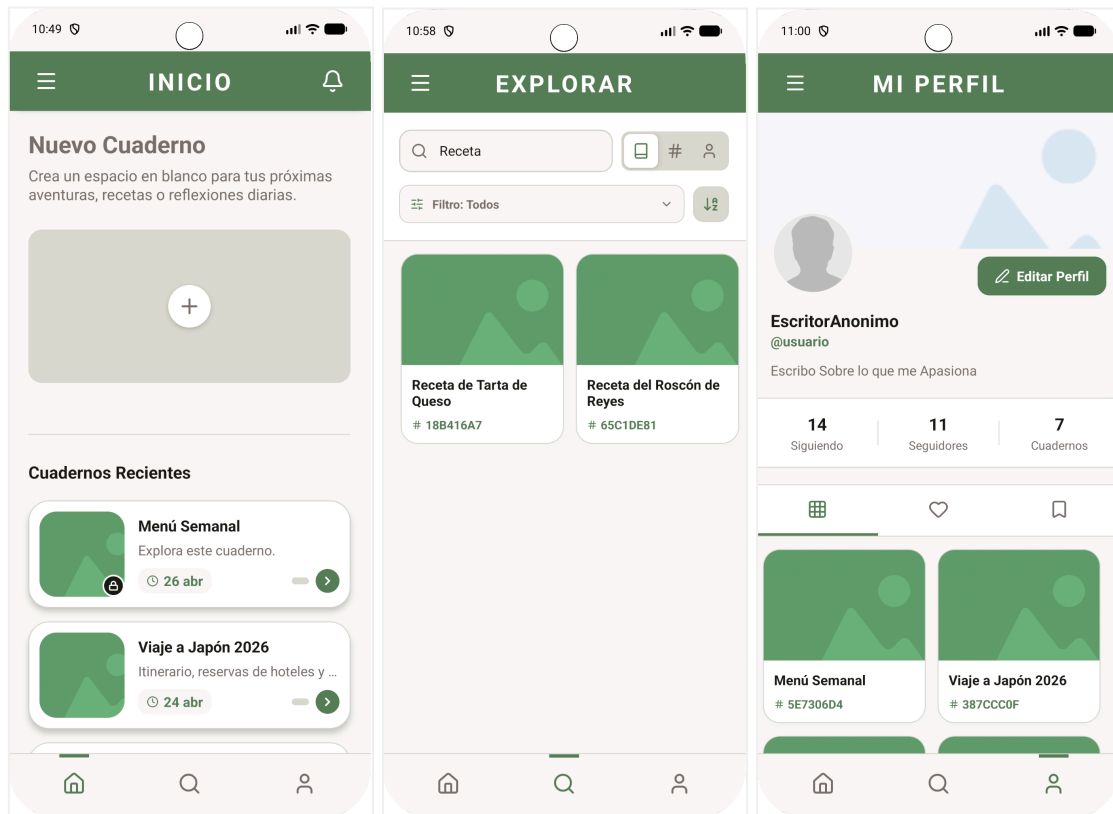
3.1.2. Inicio

Actúa como el nexo central de la aplicación. Desde esta vista, el usuario tiene acceso directo a las tres áreas fundamentales:

- **Inicio:** Núcleo de la aplicación que permite acceder al sistema de creación de cuadernos nuevos y edición de recientes así como servir como la presentación tras el registro
- **Explorar:** Sistema de búsqueda que permite navegar por todos los cuadernos y usuarios públicos buscando por nombre y en el caso de los cuadernos por código de

búsqueda, de esa. También cuenta con un filtro por etiqueta y una ordenación alfabética.

- **Mi Perfil:** Expositor de cuadernos propio donde se pueden ver información pública de cada uno, así como sus estadísticas, likes y guardados.

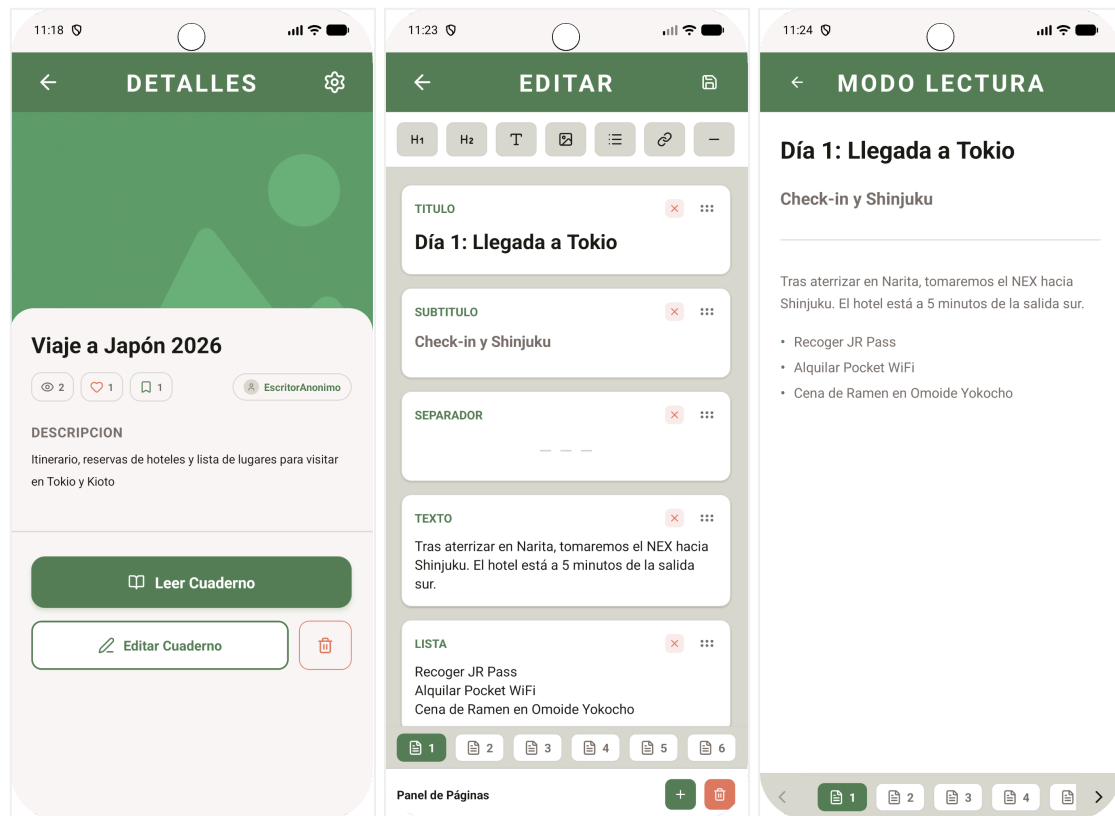


Ilustraciones 5-7 : Módulo de Inicio

3.1.3. Cuadernos

Al interactuar con un cuaderno, el sistema da una vista previa del cuaderno (Resumen Cuaderno) que se encarga de mostrar una versión resumida de los datos además de dar acceso a diferentes vistas dependiendo del rol del usuario que esté accediendo:

- **Propietario:** Tiene permiso total sobre el Editor (Modificar el contenido), Configuración (Editar título, descripción, etiquetas y portada) y Borrado (Soft delete del cuaderno) del cuaderno de su propiedad.
- **Visitante:** Puede navegar por las páginas, darle like o guardarlo.

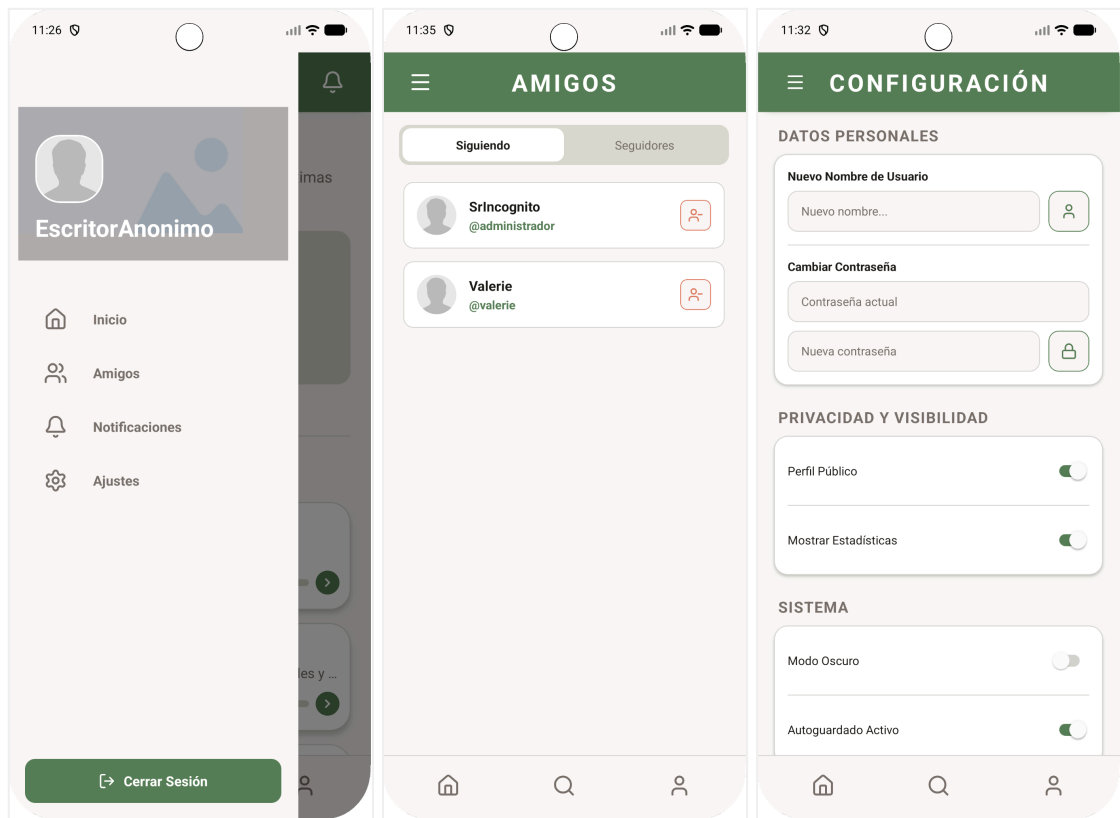


Ilustraciones 8-10 : Módulo de Cuaderno

3.1.4. Menú Lateral

A través de un menú lateral o desplegable, se cuenta con un acceso rápido a las siguientes vistas:

- **Notificaciones:** Alertas de actividad (Likes, Guardados, Relaciones, Sistema...).
- **Lista de Amigos:** Gestión de relaciones (Siguiendo y Seguidores) permitiendo acceder a sus perfiles así como eliminar la relación.
- **Configuración:** Ajustes globales de la cuenta así como la opción de cerrar sesiones o borrar la cuenta (Soft Delete del usuario).



Ilustraciones 11-13 : Menú Lateral

3.2. Modelo de base de datos

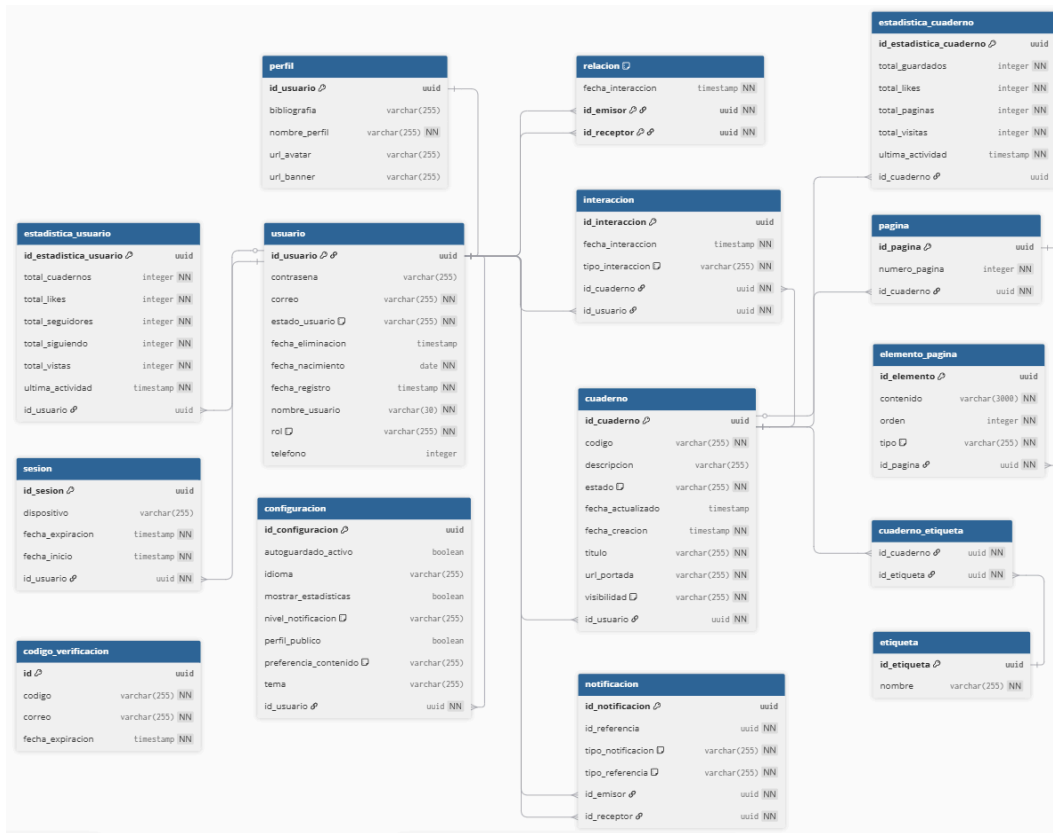


Ilustración 14 : Mapa de Base de Datos. Fuente: dbdiagram.io

Módulo de Usuario

- **Usuario:** Es la entidad central y el punto de anclaje del sistema. Almacena credenciales, estado de cuenta (ACTIVO, PENDIENTE, BANEADO, BORRADO) y el rol del usuario. Gestiona la integridad de los accesos y el control de edad.
- **Perfil:** Separa la información (biografía, avatar, banner) de los datos sensibles de la cuenta, permitiendo actualizaciones de identidad sin afectar la entidad principal.
- **Configuración:** Gestiona las preferencias de privacidad, interfaz y comportamiento del sistema (idioma, tema, notificaciones). Al ser una tabla independiente vinculada al usuario, permite una escalabilidad superior a los antiguos campos integrados.
- **Sesión:** Controla el acceso por dispositivo y la seguridad de la cuenta, gestionando la caducidad de los tiempos de acceso y la trazabilidad de las conexiones.
- **Código_Verificación:** Entidad técnica para procesos de seguridad y validación de identidad (correo) mediante tokens temporales con fecha de expiración.

Módulo de Contenido

- **Cuaderno:** Representa el eje del contenido creativo. Incluye metadatos, estado de visibilidad (PUBLICO/PRIVADO) y un código de identificación único. Es la entidad padre de toda la jerarquía de creación.
- **Página:** Define la estructura interna de los cuadernos. Utiliza un sistema de numeración secuencial para organizar el flujo de lectura del contenido.
- **Elemento_Pagina:** Contenido granular y flexible. Almacena distintos tipos de objetos (títulos, imágenes, listas, enlaces) mediante un sistema de orden y un campo de contenido extenso para evitar las limitaciones de versiones anteriores.
- **Etiqueta:** Descriptores globales únicos que sustituyen a las antiguas categorías, permitiendo una organización taxonómica transversal en toda la plataforma.
- **Cuaderno_Etiqueta:** Permite que un cuaderno sea clasificado con múltiples etiquetas.

Módulo Social

- **Relación:** Gestiona el sistema de relaciones entre usuarios (seguidores y siguiendo). Utiliza una estructura optimizada con claves compuestas para registrar el vínculo entre un emisor y un receptor, eliminando la redundancia de datos.
- **Interacción:** Registra cada acción individual de los usuarios sobre el contenido (LIKE, GUARDADO, VISTA). Esta granularidad permite generar feeds de actividad precisos y analizar tendencias de consumo.
- **Notificación:** Sistema de alertas unificado que vincula a un emisor con un receptor. Detalla el tipo de evento (social o de sistema) y mantiene una referencia directa al objeto que originó la notificación.

Módulo de Estadísticas

- **Estadística_Usuario:** Centraliza las métricas de popularidad y actividad global del usuario (total de seguidores, seguidos, vistas y cuadernos creados), optimizando las consultas de perfil sin procesar cálculos en tiempo real.

- **Estadística_Cuaderno:** Registra el rendimiento específico de cada obra. Mide el impacto mediante el conteo de likes, guardados y visitas totales, además de llevar el control del volumen de páginas y la última actividad registrada.

Tabla de Relaciones

TABLA ORIGEN	CAMPO	TABLA DESTINO	RELACIÓN
perfil	id_usuario	usuario	1:1
configuracion	id_usuario	usuario	1:1
sesion	id_usuario	usuario	N:1
cuaderno	id_usuario	usuario	N:1
pagina	id_cuaderno	cuaderno	N:1
elemento_pagina	id_pagina	pagina	N:1
cuaderno_etiqueta	id_cuaderno	cuaderno	N:M
cuaderno_etiqueta	id_etiqueta	etiqueta	N:M
relacion (Emisor)	id_emisor	usuario	N:1
relacion (Receptor)	id_receptor	usuario	N:1
interaccion	id_usuario	usuario	N:1
interaccion	id_cuaderno	cuaderno	N:1
notificacion	id_receptor	usuario	N:1

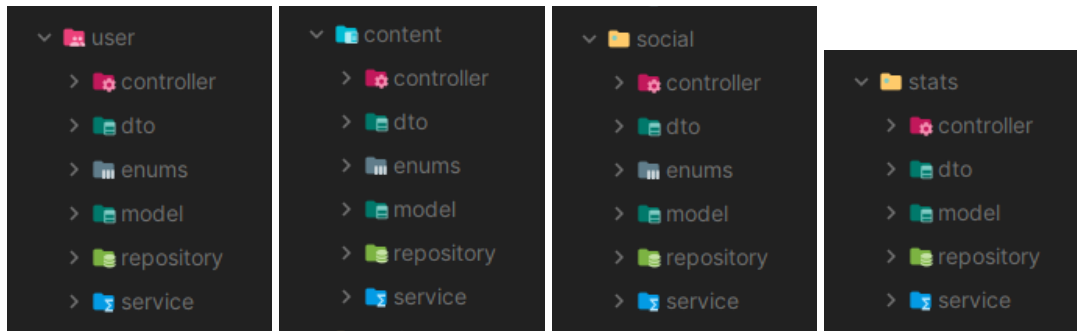
notificacion	id_emisor	usuario	N:1
estadistica_usuario	id_usuario	usuario	1:1
estadistica_cuaderno	id_cuaderno	cuaderno	1:1

Tabla 1 : Relaciones

3.3. Diseño de la arquitectura

El funcionamiento global del sistema se rige por un flujo de datos que recorre las siguientes etapas:

- **Capa de Presentación (Front-End):** Se basa en un catálogo modular donde el contenido multimedia, estilos y lógica se aplican sobre las Screens (pantallas principales) para permitir la entrada y salida de información por parte del usuario.
- **Capa de Navegación:** Las pantallas son administradas por los Navigators, que controlan el flujo de usuario en cada una de las vistas.
- **Capa de Servicios de Datos:** Desde las pantallas se invoca a los servicios de datos encargados de empaquetar la información necesaria para cada operación.
- **Capa de Comunicación y Seguridad:** Estos servicios son gestionados por el ApiService, que actúa como puente único con el servidor. Este componente se encarga de inyectar el token JWT en las cabeceras de cada petición para asegurar la identidad del usuario.
- **Capa de Control (Back-End):** El Controller del servidor recibe la petición HTTP, valida la estructura de los datos recibidos y autoriza la operación.
- **Capa de Negocio y Datos:** Una vez validada, el controlador delega la ejecución en la lógica de negocio, que finalmente realiza la apertura de la base de datos para persistir o recuperar la información en el sistema PostgreSQL.



Ilustraciones 15-18 : Estructura modular de la [API](#)

3.4. Lenguajes utilizados

- **React Native:** Framework utilizado para la creación del Front-End de la app móvil, usando JavaScript (JS) para el funcionamiento de la lógica y CSS para el diseño.
- **PostgreSQL (SQL):** Base de datos encargada de manejar la persistencia de datos así como la interacción con estos mismo. Se ha usado PostgreSQL por su mayor flexibilidad además de su manejo para los UUID.

4. DISEÑO TÉCNICO

Los siguientes apartados describen con más detalle y con descripción técnica como se ha construido el completo de la aplicación.

4.1. Libro de estilo

4.1.1. Rejilla utilizada

El sistema visual de la aplicación se basa en una **unidad de rejilla de 8px**, lo que garantiza una alineación proporcional en todos los elementos de la interfaz.

- **Unidad Base (8px):** Todos los márgenes y dimensiones son múltiplos de este valor.
- **Márgenes (Padding):** Se establece un padding raíz de 16px para los contenedores principales, asegurando un área de respiración adecuada respecto a los bordes del dispositivo.
- **Radios de Curvatura:**
 - **Botones:** 8px para un aspecto moderno y equilibrado.

- **Imágenes y Contenedores:** 12px para suavizar las esquinas de elementos visuales de mayor tamaño.
- **Bordes:** Grosor estándar de 1px para mantener la ligereza visual en las delimitaciones de componentes.

```
// — Rejilla y Métricas —  
46 references  
export const Metricas = {  
  base: 8,  
  paddingRaiz: 16,  
  radioBoton: 8,  
  radioImagen: 12,  
  bordeAncho: 1  
};
```

Ilustración 19 : Rejilla y Métricas de [AppTheme.js](#)

4.1.2. Logotipo

El logotipo está formado por una pluma y un globo terráqueo que representa la función de la aplicación de escribir y poder compartir con cualquiera, así como su nombre Hanechi que es una palabra formada por las palabras del japonés Hane (Pluma) y Chi (Tierra).

El logo será visible tanto en la página de Bienvenida como en todo el progreso de acceso a la aplicación (Inicio Sesión/Registro y Verificación Mfa)



Ilustración 20 : Logotipo

4.1.3. Tipografía

La familia tipográfica utilizada en toda la aplicación es Inter y los tamaños seleccionados son los siguientes:

NIVEL	TAMAÑO (PX)	USO
título	28	Encabezados de pantalla
subtítulo	20	Títulos de secciones y modales
normal	18	Etiquetas de navegación y botones
básico	16	Cuerpo de texto y párrafos
detalles	14	Metadatos y textos secundarios

Tabla 2: Tipografía de AppTheme.js

4.1.4. Paleta de colores

Los colores se han seleccionado para contar con un contraste apto para todos y una familiar y relajada.

Colores de Fondo y Superficie

- **Fondo Base (#FAF9F6):** Fondo principal.
- **Fondo Medio (#DAD7CD):** Fondos secundarios o secciones diferenciadas.
- **Superficie (#FFFFFF):** Tarjetas y elementos interactivos.
- **Bordes y Sombras:** #D6D3D1 para delimitaciones y #E7E5E4 para efectos de profundidad.

Colores de Marca y Acento

- **Primario (#588157):** Identidad principal y elementos de éxito.
- **Acento (#E07A5F):** Color terracota para llamadas a la acción y errores.

Colores de Texto

- **Principal (#1C1917):** Contenido de mayor importancia.

- **Secundario (#78716C):** Para información de apoyo o menos relevante.
- **Invertido (#FFFFFF):** Texto sobre fondos de color oscuro.
- **Enlace (#588157):** Indicador visual de interactividad en texto.

4.1.5. Imágenes y gráficos

- **Iconografía:** Estilo lineal con trazo geométrico uniforme.
- **Esquinas:** Redondeado estándar para portadas y elementos multimedia.
- **Profundidad:** Se aplican dos niveles de elevación nativa:
- **Sombra Sutil:** Elevation 2 (para elementos de superficie).
- **Sombra Elevada:** Elevation 4 (para elementos destacados o flotantes).

4.1.6. Estilo de componentes

Cajas de texto y etiquetas

- **Estilo:** Fondo blanco con bordes de 1px en color (**#D6D3D1**).
- **Foco:** Resaltado visual en color Acento (**#E07A5F**) para guiar al usuario.

Botones

- **Primarios:** Fondo en color primario (**#588157**) con texto invertido.
- **Acciones Críticas:** Uso del color de **acento** (**#E07A5F**) para destacar la importancia.

Mensajes de estado

La aplicación utiliza un sistema estilo de semáforo para el manejo de retroalimentación:

- **Éxito (#588157):** Confirmación de guardado o envío.
- **Error: (#E07A5F):** Validaciones fallidas o fallos de red.
- **Alerta: (#DAD7CD):** Avisos preventivos.

```

// — Paleta de colores —
93 references
export const Colores = {
  // Colores de Marca y Acento
  primario: '#588157',
  acento: '#E07A5F',

  // Colores de Fondo y Superficie
  fondoBase: '#FAF9F6',
  fondoMedio: '#DAD7CD',
  superficie: '#FFFFFF',
  bordes: '#D6D3D1',
  sombras: '#E7E5E4',

  // Colores de Texto
  texto: {
    principal: '#1C1917',
    secundario: '#78716C',
    invertido: '#FFFFFF',
    enlace: '#588157'
  },

  // Mensajes de estado
  estados: {
    exito: '#588157',
    error: '#E07A5F',
    alerta: '#DAD7CD'
  }
};

```

Ilustración 21 : Paleta de colores de [AppTheme.js](#)

4.2. Definición de componentes

4.2.1. Gestión de Acceso y Seguridad

Es el responsable de otorgar una seguridad íntegra y evitar vulnerabilidades en el ecosistema. Protegiendo la información de los usuarios.

- **Autenticación (MFA):** [AutenticacionService.js](#) maneja un flujo de seguridad en dos pasos. Tras la validación inicial de credenciales mediante el endpoint [/auth/login](#), el sistema requiere una verificación adicional a través de un código temporal enviado por correo electrónico ([/auth/mfa/verificar](#)). Y solo después de esta validación se mandael token JWT fianl.
- **Comunicación:** [ApiService.js](#) cuenta con un middleware que utiliza interceptores de [Axios](#), recuperando el token de seguridad desde el [AsyncStorage](#) e inyectando la cabecera [Authorization: Bearer](#) en cada petición. Además, revisa las respuestas del servidor para ver si ocurren errores ([401](#) y [403](#)), invalidando las sesiones expiradas.

- **Conexiones:** registrarSesion identifica de forma automática la plataforma de origen (iOS o Android) haciendo uso de la librería Platform de React Native. Esta información persiste en el servidor para ofrecer al usuario un control total sobre sus conexiones activas.

4.2.2. Gestión de Contenido y Edición

Se encarga del control, edición, persistencia y organización de la estructura jerárquica de los cuadernos.

- **Edición y Lienzo:** CuadernoDataService.js permite una modificación del contenido. En lugar de procesar las páginas completas, el sistema gestiona todo por bloques mediante el método guardarElemento. Cada nuevo contenido se envía en un DTO al backend que incluye el contenido y su índice de orden para su renderizado.
- **Persistencia:** La creación de nuevas hojas en el frontend dispara llamadas al servicio de páginas, vinculadas mediante UUIDs al cuaderno padre. Permitiendo que la jerarquía Cuaderno -> Página -> Elemento se mantenga siempre sincronizada.

4.2.3. Gestión de Perfiles

Componente destinado a la administración de la identidad digital y la personalización de la experiencia de usuario.

- **Identidad:** A través de UsuarioDataService.js, el sistema diferencia técnicamente entre los datos de cuenta (sensibles) y los datos de perfil (estéticos). Las actualizaciones se realizan mediante peticiones de tipo PATCH al endpoint `/perfiles`, permitiendo modificar elementos como la biografía o el avatar de forma independiente a las credenciales de acceso.
- **Configuración y Visibilidad:** El componente gestiona la entidad Configuración, permitiendo persistir las preferencias de privacidad (perfil público/privado) y comportamiento del sistema (modo oscuro, autoguardado).
- **Ciclo de Vida del Usuario:** Se implementa la lógica de "Soft Delete" a través del método `eliminarCuenta`, lo que permite marcar al usuario como borrado en el sistema sin comprometer la integridad de las relaciones históricas en la base de datos.

4.2.4. Gestión Social y de Estadísticas

Módulo encargado de la capa comunicativa y la monitorización de impacto del contenido en la comunidad.

- **Relaciones y Notificaciones:** El servicio `SocialDataService.js` gestiona el sistema de seguidores mediante una estructura de emisor y receptor. Cada acción social relevante dispara una notificación en el servidor que es consumida por el frontend para alimentar el feed de alertas de actividad del usuario.
- **Interacciones y Feedback:** Mediante `InteraccionDataService.js`, se registran eventos asíncronos de tipo `"Like"`, `"Guardado"` o `"Vista"`. Estas acciones alimentan el motor de descubrimiento de la aplicación, permitiendo al usuario filtrar sus colecciones personales.
- **Optimización de Métricas:** Para evitar el procesamiento de cálculos complejos en tiempo real, el sistema consulta las entidades `Estadistica_Usuario` y `Estadistica_Cuaderno`. Estos componentes recuperan valores consolidados, optimizando el rendimiento de las vistas de perfil y los listados de exploración.

4.2.5. Estado Global (Zustand)

Para gestionar la complejidad de la aplicación y evitar la redundancia de datos entre pantallas, se han implementado "Stores" centralizados que actúan como la única fuente de verdad.

- **Persistencia (`AuthStore`):** El componente `AuthStore.js` gestiona la sesión del usuario. Implementa una función de `inicializarAuth` que recupera el token y los datos del usuario desde el almacenamiento persistente (`AsyncStorage`) al arrancar la app, permitiendo mantener la sesión activa sin re-autenticación.
- **Control del Ciclo de Vida del Lienzo (`CuadernoStore`):** Este componente centraliza la lógica de edición. Gestiona la carga de páginas, el reordenamiento de bloques y el control de `estado sucio`. Utiliza un sistema de `identificadores temporales` para manejar nuevos elementos en la interfaz antes de que el servidor asigne un UUID tras la sincronización.

4.2.6. Servicio de Notificación MFA vía EmailService

Este servicio consta de la MFA mediante la entrega de códigos OTP (One-Time Password) directamente en el buzón de correo del usuario.

El flujo comienza en la capa de negocio (AutenticacionService). Cuando un usuario supera la validación de contraseña cifrada, el servicio dispara el método enviarCodigo.

1. **Generación de Cripto-PIN:** Se genera una cadena de números aleatorio de 6.
2. **Persistencia Temporal:** El código se guarda en la tabla codigo_verificacion vinculado al correo electrónico del usuario, asignándole una vida útil de 5 minutos para evitar ataques de fuerza bruta.
3. **Invocación del Servicio de Correo:** Se delega la tarea al EmailService, pasando el destino y el código generado.

El EmailService utiliza la interfaz JavaMailSender de Spring Boot para comunicarse con el servidor de correo configurado.

- **Mensajería MIME:** El sistema genera objetos MimeMessage que permiten el envío de contenido HTML para ofrecer una vista más compleja al usuario.
- **Codificación y Soporte:** Se configura un MimeMessageHelper con codificación UTF-8 para asegurar que todos los caracteres y símbolos se visualicen correctamente en cualquier cliente de correo (Gmail, Outlook, etc.).

La lógica de presentación del correo no es estática; se genera dinámicamente mediante el método generarCuerpoHtml, donde se utiliza el método .formatted(codigo) de Java para insertar el PIN de seguridad dentro del HTML.

Una vez que el correo es entregado, la aplicación móvil (Verificacion2FAScreen.js) actúa como receptor de esta información.

- **Validación de Red:** Si el usuario no recibe el correo, una función permite el reenvío, invalidando el antiguo código y reiniciando el proceso con uno nuevo.

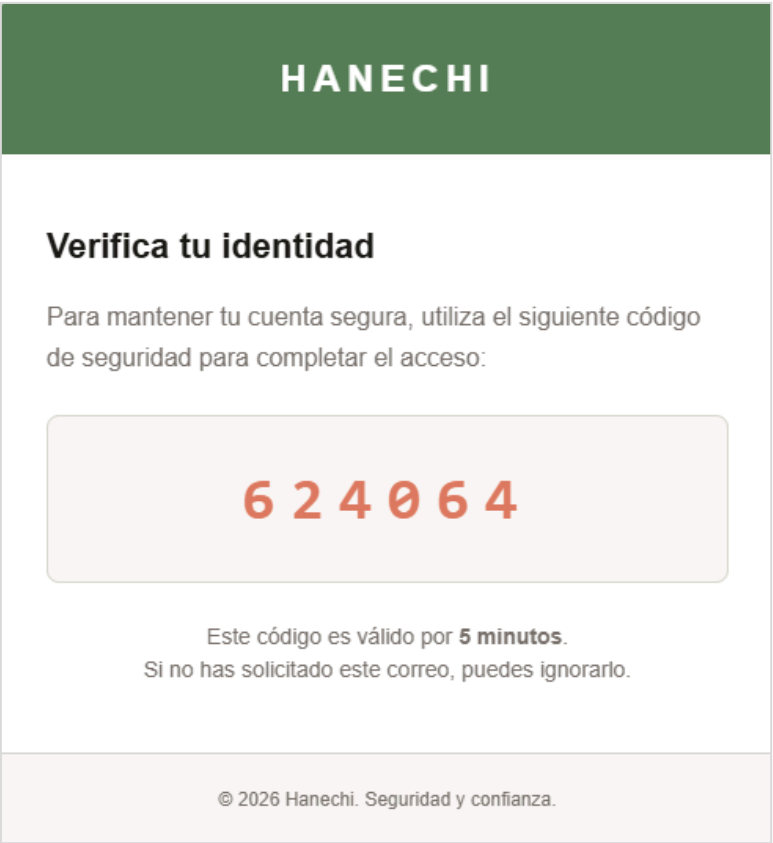


Ilustración 22 : Correo Verificación

4.3. Diseño de tablas de la base de datos

Nombre Tabla	Atributo	Descripción	Tipo de Dato	Regla Validación	NULL / NOT NULL	PK / FK	Tabla Referencia
usuario	id_usuario	ID único del usuario	UUID		NOT NULL	PK	
	nombre_usuario	Nombre de cuenta	VARCHAR(30)	Longitud máx 30	NOT NULL		
	correo	Email vinculado	VARCHAR(255)	Formato email / Único	NOT NULL		
	contrasena	Hash de contraseña	VARCHAR(255)		NULL		
	estado_usuario	Estado de la cuenta	VARCHAR(255)	ACTIVO, BORRADO, BANNEADO, PENDIENTE USER, ADMIN	NOT NULL		
	rol	Permisos del sistema	VARCHAR(255)		NOT NULL		
	fecha_nacimiento	Fecha de nacimiento	DATE		NOT NULL		
	fecha_registro	Alta en el sistema	TIMESTAMPZ		NOT NULL		
	fecha_eliminacion	Soft delete	TIMESTAMPZ		NULL		
perfil	telefono	Teléfono de contacto	INTEGER		NULL		
	id_usuario	ID de usuario	UUID		NOT NULL	PK / FK	usuario
	nombre_perfil	Nombre público	VARCHAR(255)		NOT NULL		
	biografia	"Sobre mí"	VARCHAR(255)		NULL		
	url_avatar	Foto de perfil	VARCHAR(255)		NULL		
configuracion	url_banner	Foto de portada	VARCHAR(255)		NULL		
	id_configuracion	ID configuración	UUID		NOT NULL	PK	
	id_usuario	Propietario	UUID	Unique	NOT NULL	FK	usuario
	autoguardado_activo	Status autoguardado	BOOLEAN		NULL		
	idioma	Idioma interfaz	VARCHAR(255)		NULL		
	mostrar_estadisticas	Visibilidad stats	BOOLEAN		NULL		
	nivel_notificacion	Filtro avisos	VARCHAR(255)	SIGUIENDO, AMIGOS, TODOS	NULL		
	perfil_publico	Privacidad perfil	BOOLEAN		NULL		
	preferencia_contenido	Filtro de edad	VARCHAR(255)	TODO_PUBLICO, MAS_16, PARA_ADULTOS	NULL		
sesion	id_sesion	ID de sesión	UUID		NOT NULL	PK	
	id_usuario	Usuario	UUID		NOT NULL	FK	usuario
	dispositivo	Info hardware	VARCHAR(255)		NULL		
	fecha_inicio	Login	TIMESTAMPZ		NOT NULL		
	fecha_expiracion	Logout forzado	TIMESTAMPZ		NOT NULL		

Tabla 3: Módulo de Estadísticas. Fuente: [Base de Datos \(Usuario\)](#)

Nombre Tabla	Atributo	Descripción	Tipo de Dato	Regla Validación	NULL / NOT NULL	PK / FK	Tabla Referencia
cuaderno	id_cuaderno	ID del cuaderno	UUID		NOT NULL	PK	
	id_usuario	Autor	UUID		NOT NULL	FK	usuario
	titulo	Título del cuaderno	VARCHAR(255)		NOT NULL		
	descripcion	Síntesis corta	VARCHAR(255)		NULL		
	codigo	Código único/slug	VARCHAR(255)		NOT NULL		
	url_portada	Imagen de portada	VARCHAR(255)		NOT NULL		
	estado	Estado integridad	VARCHAR(255)	ACTIVO, BORRADO, BANNADO	NOT NULL		
	visibilidad	Nivel de acceso	VARCHAR(255)	PUBLICO, PRIVADO	NOT NULL		
	fecha_creacion	Fecha creación	TIMESTAMPZ		NOT NULL		
pagina	fecha_actualizado	Último cambio	TIMESTAMPZ		NULL		
	id_pagina	ID de la página	UUID		NOT NULL	PK	
	id_cuaderno	Cuaderno padre	UUID		NOT NULL	FK	cuaderno
	numero_pagina	Orden secuencial	INTEGER		NOT NULL		
colaborador	id_elemento	ID del bloque	UUID		NOT NULL	PK	
	id_pagina	Página contenedora	UUID		NOT NULL	FK	pagina
	contenido	Datos del elemento	VARCHAR(3000)		NOT NULL		
	orden	Posición en página	INTEGER		NOT NULL		
	tipo	Categoría bloque	VARCHAR(255)	EXTO, IMAGEN, SUBTITULO, ENLACE, LISTA, SE	NOT NULL		
etiqueta	id_etiqueta	ID etiqueta	UUID		NOT NULL	PK	
	nombre	Nombre único	VARCHAR(255)	Unique	NOT NULL		
cuaderno_etiqueta	id_cuaderno	Relación cuaderno	UUID		NOT NULL	FK	cuaderno
	id_etiqueta	Relación etiqueta	UUID		NOT NULL	FK	etiqueta

Tabla 4: Módulo de Estadísticas. Fuente: [Base de Datos \(Contenido\)](#)

Nombre Tabla	Atributo	Descripción	Tipo de Dato	Regla Validación	NULL / NOT NULL	PK / FK	Tabla Referencia
relacion	id_emisor	Seguidor	UUID		NOT NULL	PK / FK	usuario
	id_receptor	Seguido	UUID	I= id_emisor	NOT NULL	PK / FK	usuario
	fecha_interaccion	Inicio relación	TIMESTAMPZ		NOT NULL		
interaccion	id_interaccion	ID acción	UUID		NOT NULL	PK	
	id_usuario	Usuario que actúa	UUID		NOT NULL	FK	usuario
	id_cuaderno	Objeto actuado	UUID		NOT NULL	FK	cuaderno
	tipo_interaccion	Tipo de feedback	VARCHAR(255)	LIKE, GUARDADO, VISTA	NOT NULL		
	fecha_interaccion	Fecha evento	TIMESTAMPZ		NOT NULL		
notificacion	id_notificacion	ID alerta	UUID		NOT NULL	PK	
	id_receptor	Destinatario	UUID		NOT NULL	FK	usuario
	id_emisor	Autor acción	UUID		NOT NULL	FK	usuario
	id_referencia	ID objeto origen	UUID		NOT NULL		
	tipo_notificacion	Categoría alerta	VARCHAR(255)	LIKE, GUARDADO, FOLLOW, SISTEMA	NOT NULL		
	tipo_referencia	Entidad origen	VARCHAR(255)	USUARIO, CUADERNO, SISTEMA	NOT NULL		

Tabla 5: Módulo de Estadísticas. Fuente: [Base de Datos \(Social\)](#)

Nombre Tabla	Atributo	Descripción	Tipo de Dato	Regla Validación	NULL / NOT NULL	PK / FK	Tabla Referencia
estadistica_usuario	id_estadistica_usuario	ID estadística	UUID		NOT NULL	PK	
	id_usuario	Usuario vinculado	UUID	Unique	NULL	FK	usuario
	total_seguidores	Contador	INTEGER	>= 0	NOT NULL		
	total_siguiendo	Contador	INTEGER	>= 0	NOT NULL		
	total_cuadernos	Contador	INTEGER	>= 0	NOT NULL		
	total_likes	Likes recibidos	INTEGER	>= 0	NOT NULL		
	total_vistas	Vistas recibidas	INTEGER	>= 0	NOT NULL		
	ultima_actividad	Fecha log	TIMESTAMPZ		NOT NULL		
estadistica_cuaderno	id_estadistica_cuaderno	ID estadística	UUID		NOT NULL	PK	
	id_cuaderno	Cuaderno vinculado	UUID	Unique	NULL	FK	cuaderno
	total_guardados	Contador	INTEGER	>= 0	NOT NULL		
	total_likes	Contador	INTEGER	>= 0	NOT NULL		
	total_paginas	Contador	INTEGER	>= 0	NOT NULL		
	total_visitas	Contador	INTEGER	>= 0	NOT NULL		
	ultima_actividad	Fecha log	TIMESTAMPZ		NOT NULL		

Tabla 6: Módulo de Estadísticas. Fuente: [Base de Datos \(Estadísticas\)](#)

5. DEMOSTRACIÓN

5.1. Estructura del Proyecto

- **/backend:** Proyecto Spring Boot (Java 21). Configurado con ddl-auto=update para la generación automática de tablas mediante Hibernate.
- **/database:** Contiene el script data.sql necesario para la inserción de etiquetas y datos predeterminados.
- **/frontend:** Aplicación móvil desarrollada con React Native y Expo.

5.2. Requisitos del Entorno

Para la ejecución del proyecto, se necesita:

- **Java JDK 21 y Node.js (LTS).**
- **PostgreSQL 15+** (recomendado mediante Docker).
- **Expo Go** instalado en un dispositivo móvil o un emulador configurado.

5.3. Configuración y Puesta en Marcha

Paso 1: Clonar el repositorio

En la carpeta deseada, ejecuta los siguientes comandos:

- git clone <https://github.com/AlejandroDastard/Hanechi-Notebook.git>
- cd Hanechi-Notebook

Paso 2: Base de Datos y Backend

1. **Levantar Base de Datos:** Crear un contenedor en Docker de PostgreSQL o asegurar una instancia local corriendo en el puerto 5432.
2. **Crear base de datos:** Crear una base de datos vacía llamada hanechi_notebook.
3. **Configurar Entornos:**

- Localizar el archivo .env.template en /backend.
 - Renombrarlo a .env y completar los datos de conexión y correo (MFA).
4. **Iniciar Servidor:** En /backend y ejecutar los comandos:
- `docker compose up -d`
 - `mvn clean install`
 - `mvn spring-boot:run`
`-Dspring-boot.run.arguments="--spring.jpa.hibernate.ddl-auto=create`
`--spring.sql.init.mode=always"`

Al arrancar por primera vez, Spring Boot creará automáticamente la estructura de tablas.

Paso 3: Inserción de Datos Iniciales

Una vez que el backend haya arrancado y las tablas existan en PostgreSQL:

1. Ejecutar el script database/data.sql para cargar las etiquetas predeterminadas.

Paso 4: Configuración del Frontend

1. **Acceder a la carpeta /frontend e instalar dependencias:** npm install
2. **Configurar Conexión:** Crear un nuevo archivo .env, copiando el de .env.template del frontend y ajustar la variable EXPO_PUBLIC_API_URL con la IP del servidor.
3. **Iniciar Aplicación:** npx expo start

6. BIBLIOGRAFÍA

- **Canvas:** Contenido dado en clase durante el curso.
- **Axios:** Cliente HTTP para la ejecución de peticiones y manejo de comunicación con APIs externas.
- **Zustand:** Gestión de estado global basada en hooks.
- **Async Storage:** Sistema de almacenamiento local persistente de datos.

- [Expo Image Picker](#): Integración de selección de imágenes y acceso a la cámara.
- [React Navigation](#): Gestión de rutas y navegación (Stack, Tabs y Drawer) entre pantallas.
- [React Native Draggable FlatList](#): Creación de listas con reordenamiento mediante gestos.
- [Lucide React Native](#): Implementación y personalización de iconos vectoriales.
- [Jakarta Mail Documentation](#): Envío de mensajes de correo electrónico.
- [Spring Framework Mail Support](#): Envío de correos electrónicos.

7. ANEXOS

7.1. Código fuente

7.2.1. Backend (/backend)

Raíz del Backend (/src/main/java/com/hanechi/api)

- **AplicacionHanechiApi.java**: Clase principal que contiene el método que arranca el servidor.
- **POM**: Configuración de dependencias y funcionamiento de la API.

Configuración e Infraestructura (/infrastructure)

- **config/ApplicationConfiguration.java**: Define los beans de configuración global, como el codificador de contraseñas ([BCryptPasswordEncoder](#)).
- **config/SecurityConfiguration.java**: Establece las reglas de seguridad HTTP, rutas públicas/privadas y sesiones.
- **config/WebMvcConfig.java**: Configuración del sistema de subidas de imágenes.
- **mail/EmailService.java**: Servicio encargado de enviar correos electrónicos mediante [SMTP](#).
- **controller/ImageUploadController.java**: Endpoint para la subida de imágenes

Seguridad y JWT (/security/jwt)

- **JwtAuthenticationFilter.java:** Filtro interceptor que extrae y valida el token JWT de las cabeceras HTTP.
- **JwtService.java:** Componente utilitario para la generación, firma criptográfica y validación de expiración de tokens.

Módulo de Contenido (/modules/content)

- **/controller**
 - **CuadernoController.java:** Endpoints para gestionar las peticiones de la entidad Cuaderno.
 - **ElementoPaginaController.java:** Endpoints para gestionar las peticiones de la entidad ElementoPagina.
 - **EtiquetaController.java:** Endpoints para gestionar las peticiones de la entidad Etiqueta.
 - **PaginaController.java:** Endpoints para gestionar las peticiones de la entidad Pagina.
- **/dto**
 - **ActualizacionCuadernoDTO.java:** DTO de entrada para modificar la entidad Cuaderno.
 - **CuadernoDTO.java:** DTO principal para la transferencia de la entidad Cuaderno.
 - **ElementoPaginaDTO.java:** DTO principal para la transferencia de la entidad ElementoPagina.
 - **EtiquetaDTO.java:** DTO principal para la transferencia de la entidad Etiqueta.
 - **PaginaDTO.java:** DTO principal para la transferencia de la entidad Pagina.
 - **PeticionCuadernoDTO.java:** DTO de entrada para crear la entidad Cuaderno.
 - **PeticionPaginaDTO.java:** DTO de entrada para crear la entidad Pagina.
- **/enums**
 - **EstadoCuaderno.java:** ACTIVO, BORRADO.

- **TipoElemento.java:** TITULO, SUBTITULO, TEXTO, IMAGEN, SEPARADOR, LISTA.
- **VisibilidadCuaderno.java:** PÚBLICO, PRIVADO
- **/model**
 - **Cuaderno.java:** Representación de la tabla Cuaderno en la base de datos.
 - **ElementoPagina.java:** Representación de la tabla ElementoPagina en la base de datos.
 - **Etiqueta.java:** Representación de la tabla Etiqueta en la base de datos.
 - **Pagina.java:** Representación de la tabla Pagina en la base de datos.
- **/model/keys**
 - **ClaveCuadernoEtiqueta.java:** Clave primaria compuesta para la relación entre Cuaderno y Etiqueta.
- **/repository**
 - **CuadernoRepository.java:** Operaciones de acceso a datos para la tabla Cuaderno.
 - **ElementoPaginaRepository.java:** Operaciones de acceso a datos para la tabla ElementoPagina.
 - **EtiquetaRepository.java:** Operaciones de acceso a datos para la tabla Etiqueta.
 - **PaginaRepository.java:** Operaciones de acceso a datos para la tabla Pagina.
- **/service**
 - **CuadernoService.java:** Lógica de negocio y reglas de dominio para la entidad Cuaderno.
 - **ElementoPaginaService.java:** Lógica de negocio y reglas de dominio para la entidad ElementoPagina.
 - **EtiquetaService.java:** Lógica de negocio y reglas de dominio para la entidad Etiqueta.
 - **FabricaCuadernoService.java:** Mapeo manual mediante constructores entre la entidad Cuaderno, su DTO y llamar a FabricaEstadisticasCuadernoService.

- **FabricaElementoPaginaService.java:** Mapeo manual mediante constructores entre la entidad ElementoPagina y su DTO.
- **FabricaPaginaService.java:** Mapeo manual mediante constructores entre la entidad Pagina y su DTO.
- **PaginaService.java:** Lógica de negocio y reglas de dominio para la entidad Pagina.

Módulo Social (/modules/social)

- **/controller:** Rutas REST de los endpoints del módulo.
 - **InteraccionController.java:** Endpoints para gestionar las peticiones de la entidad Interaccion.
 - **NotificacionController.java:** Endpoints para gestionar las peticiones de la entidad Notificacion.
 - **RelacionController.java:** Endpoints para gestionar las peticiones de la entidad Relacion.
- **/dto**
 - **InteraccionDTO.java:** DTO principal para la transferencia de la entidad Interaccion.
 - **NotificacionDTO.java:** DTO principal para la transferencia de la entidad Notificacion.
 - **RelacionDTO.java:** DTO principal para la transferencia de la entidad Relacion.
- **/enums**
 - **TipoInteraccion.java:** LIKE, GUARDADO, VISTA
 - **TipoNotificacion.java:** LIKE, VISTA, SEGUIMIENTO, SISTEMA
 - **TipoReferencia.java:** USUARIO, CUADERNO.
- **/model**
 - **Interaccion.java:** Representación de la tabla Interaccion en la base de datos.
 - **Notificacion.java:** Representación de la tabla Notificacion en la base de datos.

- **Relacion.java:** Representación de la tabla Relacion en la base de datos.
- **/model/keys**
 - **ClaveRelacion.java:** Clave primaria compuesta para la relación entre emisores y receptores.
- **/repository**
 - **InteraccionRepository.java:** Operaciones de acceso a datos para la tabla Interaccion.
 - **NotificacionRepository.java:** Operaciones de acceso a datos para la tabla Notificacion.
 - **RelacionRepository.java:** Operaciones de acceso a datos para la tabla Relacion.
- **/service**
 - **FabricaInteraccionService.java:** Mapeo manual mediante constructores entre la entidad Interaccion y su DTO.
 - **FabricaNotificacionService.java:** Mapeo manual mediante constructores entre la entidad Notificacion y su DTO.
 - **FabricaRelacionService.java:** Mapeo manual mediante constructores entre la entidad Relacion y su DTO.
 - **InteraccionService.java:** Lógica de negocio y reglas de dominio para la entidad Interaccion.
 - **NotificacionService.java:** Lógica de negocio y reglas de dominio para la entidad Notificacion.
 - **RelacionService.java:** Lógica de negocio y reglas de dominio para la entidad Relacion.

Módulo de Estadísticas (/modules/stats)

- **/controller**
 - **EstadisticasCuadernoController.java:** Endpoints para gestionar las peticiones de la entidad EstadisticasCuaderno.

- **EstadisticasUsuarioController.java:** Endpoints para gestionar las peticiones de la entidad EstadisticasUsuario.
- **/dto**
 - **EstadisticasCuadernoDTO.java:** DTO principal para la transferencia de la entidad EstadisticasCuaderno.
 - **EstadisticasUsuarioDTO.java:** DTO principal para la transferencia de la entidad EstadisticasUsuario.
- **/model**
 - **EstadisticasCuaderno.java:** Representación de la tabla EstadisticasCuaderno en la base de datos.
 - **EstadisticasUsuario.java:** Representación de la tabla EstadisticasUsuario en la base de datos.
- **/repository**
 - **EstadisticasCuadernoRepository.java:** Operaciones de acceso a datos para la tabla EstadisticasCuaderno.
 - **EstadisticasUsuarioRepository.java:** Operaciones de acceso a datos para la tabla EstadisticasUsuario.
- **/service**
 - **EstadisticasCuadernoService.java:** Lógica de negocio y reglas de dominio para la entidad EstadisticasCuaderno.
 - **EstadisticasUsuarioService.java:** Lógica de negocio y reglas de dominio para la entidad EstadisticasUsuario.
 - **FabricaEstadisticasCuadernoService.java:** Mapeo manual mediante constructores entre la entidad EstadisticasCuaderno y su DTO.
 - **FabricaEstadisticasUsuarioService.java:** Mapeo manual mediante constructores entre la entidad EstadisticasUsuario y su DTO.

Módulo de Usuario (/modules/user)

- **/controller**

- **AutenticacionController.java:** Endpoints para gestionar las peticiones del flujo de Autenticación.
- **ConfiguracionController.java:** Endpoints para gestionar las peticiones de la entidad Configuracion.
- **PerfilController.java:** Endpoints para gestionar las peticiones de la entidad Perfil.
- **SesionController.java:** Endpoints para gestionar las peticiones de la entidad Sesion.
- **UsuarioController.java:** Endpoints para gestionar las peticiones de la entidad Usuario.

- **/dto**

- **ActualizacionConfiguracionDTO.java:** DTO de entrada para modificar la entidad Configuracion.
- **ActualizacionPerfilDTO.java:** DTO de entrada para modificar la entidad Perfil.
- **AutenticacionDTO.java:** DTO principal para la transferencia del resultado de Autenticación.
- **CambioContrasenaDTO.java:** DTO de entrada para gestionar el flujo de CambioContrasena.
- **CambioNombreDTO.java:** DTO de entrada para gestionar el flujo de CambioNombre.
- **CodigoVerificacionDTO.java:** DTO de entrada para gestionar la entidad CodigoVerificacion.
- **ConfiguracionDTO.java:** DTO principal para la transferencia de la entidad Configuracion.
- **InicioSesionDTO.java:** DTO de entrada para el flujo de InicioSesion.
- **PerfilDTO.java:** DTO principal para la transferencia de la entidad Perfil.
- **PerfilPublicoDTO.java:** DTO de salida filtrado para la entidad Perfil.
- **RegistroDTO.java:** DTO de entrada para el flujo de Registro.

- **SesionDTO.java:** DTO principal para la transferencia de la entidad Sesion.
- **UsuarioDTO.java:** DTO principal para la transferencia de la entidad Usuario.
- **/enums**
 - **EstadoUsuario.java:** PENDIENTE, ACTIVO, BANEADO, BORRADO.
 - **NivelNotificacion.java:** TODOS, AMIGOS, NADIE.
 - **PreferenciaContenido.java:** TODO_PUBLICO, MAS_16, PARA_ADULTOS.
 - **RolUsuario.java:** USER, ADMIN.
- **/model**
 - **CodigoVerificacion.java:** Representación de la tabla CodigoVerificacion en la base de datos.
 - **Configuracion.java:** Representación de la tabla Configuracion en la base de datos.
 - **Perfil.java:** Representación de la tabla Perfil en la base de datos.
 - **Sesion.java:** Representación de la tabla Sesion en la base de datos.
 - **Usuario.java:** Representación de la tabla Usuario en la base de datos.
- **/repository**
 - **CodigoVerificacionRepository.java:** Operaciones de acceso a datos para la tabla CodigoVerificacion.
 - **ConfiguracionRepository.java:** Operaciones de acceso a datos para la tabla Configuracion.
 - **PerfilRepository.java:** Operaciones de acceso a datos para la tabla Perfil.
 - **SesionRepository.java:** Operaciones de acceso a datos para la tabla Sesion.
 - **UsuarioRepository.java:** Operaciones de acceso a datos para la tabla Usuario.
- **/service**
 - **AutenticacionService.java:** Lógica de negocio y reglas de dominio para el flujo de Autenticación.

- **ConfiguracionService.java:** Lógica de negocio y reglas de dominio para la entidad Configuracion.
- **FabricaConfiguracionService.java:** Mapeo manual mediante constructores entre la entidad Configuracion y su DTO.
- **FabricaPerfilService.java:** Mapeo manual mediante constructores entre la entidad Perfil y su DTO.
- **FabricaSesionService.java:** Mapeo manual mediante constructores entre la entidad Sesion y su DTO.
- **FabricaUsuarioService.java:** Mapeo manual mediante constructores entre la entidad Usuario, su DTO y llamar a FabricaConfiguracionService, FabricaPerfilService y FabricaEstadisticasUsuarioService.
- **PerfilService.java:** Lógica de negocio y reglas de dominio para la entidad Perfil.
- **SesionService.java:** Lógica de negocio y reglas de dominio para la entidad Sesion.
- **UsuarioService.java:** Lógica de negocio y reglas de dominio para la entidad Usuario.

Recursos (/resources y raíz)

- **application.properties:** Fichero de configuración de Spring que centraliza las credenciales de base de datos, el puerto y propiedades del entorno.

7.2.2. Base de Datos (/database)

- **data.sql:** Script que se debe ejecutar durante el arranque inicial. Se encarga de poblar la base de datos con información base de las etiquetas predeterminadas.

7.2.3. Frontend (/frontend)

Raíz del Frontend

- **App.js:** Punto de entrada global que envuelve los navegadores y el contexto principal.
- **app.json:** Configuración de Expo (nombre, versión, iconos, permisos de SO).

- **package.json**: Manifiesto de npm con dependencias y scripts de ejecución.
- **package-lock.json**: Congelación del árbol de dependencias para asegurar la reproducibilidad.

Imágenes (/assets/img)

- **avatar/default_avatar.png**: Imagen por defecto para perfiles recién creados.
- **banner/default_banner.jpg**: Cabecera predeterminada para el perfil visual.
- **brand/fondo_auth.jpg**: Gráfico de fondo inmersivo para el túnel de autenticación.
- **brand/logo.png**: Logotipo vectorizado principal de la aplicación.
- **notebook/default_portada.png**: Diseño base al crear un nuevo cuaderno vacío.

Componentes de Interfaz (/components)

- **/cards**: Componentes orientados a mostrar resúmenes de información en formato de tarjeta interactiva, preparados para listas eficientes (FlatLists).
 - **BusquedaComponent.js**: Tarjeta visual compacta para resultados del menú Explorar.
 - **CuadernoComponent.js**: Tarjeta que renderiza la miniatura, título y estadísticas de una obra.
- **/common**
 - **BotonComponent.js**: Botón para la sección de acceso a la aplicación.
 - **EntradaComponent.js**: Campo de entrada de texto unificado para la sección de acceso a la aplicación con soporte para validación e iconos.
 - **EstadisticaComponent.js**: Elemento para mostrar los contadores numéricos estéticamente.
- **/layout**
 - **CabeceraComponent.js**: Barra superior dinámica con botón de retroceso y acciones contextuales.
 - **PestanasComponent.js**: Componente personalizado que estiliza el Navegador inferior.

- **/notebook**

- **BloqueComponent.js:** Componente polimórfico que se adapta para mostrar o editar texto, imágenes o listas.
- **IndiceComponent.js:** Menú flotante o modal para saltar entre las páginas de un cuaderno.
- **LienzoComponent.js:** Contenedor principal que maneja el Drag and Drop y ordena los BloqueComponent.

Enrutamiento (/navigation)

- **AutenticacionNavigator.js:** Agrupa la movilidad entre pantallas de /auth.
- **CuadernosNavigator.js:** Agrupa la movilidad entre pantallas de /notebook.
- **MenuLateralNavigator.js:** Navegador que permite entrar a Configuración y /social.
- **PestanasNavigator.js:** Agrupa la movilidad entre pantallas de /explore, /home y /profile.
- **PrincipalNavigator.js:** Enrutador maestro que decide el flujo inicial según el estado de sesión.

Pantallas de Flujo (/screens)

- **/auth**
 - **BienvenidaScreen.js:** Splash screen manual con opciones de entrada.
 - **InicioSesionScreen.js:** Formulario de login y enlaces de recuperación.
 - **Verificacion2FAScreen.js:** Interfaz para introducir el código temporal enviado por correo.
- **/explore**
 - **ExplorarScreen.js:** Muro infinito con barra de búsqueda y filtros por etiquetas.
- **/home**
 - **InicioScreen.js:** Dashboard con carrusel de obras recientes y botón de acción rápida.

- **/notebook**
 - **ConfiguracionCuadernoScreen.js:** Formulario de edición de datos.
 - **CrearCuadernoScreen.js:** Asistente rápido para instanciar una obra vacía.
 - **EditarCuadernoScreen.js:** Entorno de trabajo para el editor de bloques.
 - **ResumenCuadernoScreen.js:** Previa del cuaderno con datos y estadísticas.
 - **VerCuadernoScreen.js:** Modo lectura inmersivo a pantalla completa.
- **/profile**
 - **ConfiguracionScreen.js:** Panel de ajustes globales.
 - **EditarPerfilScreen.js:** Formulario de actualización de biografía y multimedia.
 - **PerfilScreen.js:** Vista personal con estadísticas y cuadrícula de cuadernos.
- **/social**
 - **AmigosScreen.js:** Listado separado en "Siguiendo" y "Seguidores".
 - **NotificacionesScreen.js:** Centro de alertas.
 - **PerfilPublicoScreen.js:** Vista del perfil de otra persona desde el que se puede Seguirle o acceder a sus cuadernos públicos.

Lógica (/services, /store, /theme, /utils)

- **/services:** Centraliza todas las llamadas HTTP. Encapsula las URLs y delega en Axios la inyección de la seguridad JWT.
 - **ApiService.js:** Instancia central de Axios con interceptores de tokens y control de errores.
 - **AutenticacionService.js:** Peticiones al servidor de identidad (login, registro, MFA).
 - **CuadernoDataService.js:** Peticiones para Cuadernos, Páginas, Elementos y Etiquetas.
 - **InteraccionDataService.js:** Peticiones para Intera y Estadísticas.
 - **SocialDataService.js:** Peticiones para Notificaciones, Relaciones y Etiquetas.

- **UsuarioDataService.js:** Peticiones para Usuarios, Perfiles, Configuraciones y Sesiones .
- **/store**
 - **AuthStore.js:** Almacén global de la sesión.
 - **CuadernoStore.js:** Almacén temporal de las reordenaciones del lienzo.
- **/theme**
 - **AppTheme.js:** Diccionario maestro de colores, espaciados y reglas de tipografía.
- **/utils**
 - **ImagenHelper.js:** Procesador de Imagenes.